

Проект: Управление мышью через гироскоп на Raspberry Pi

Оглавление

1. [Авторы](#)
2. [Обзор проекта](#)
3. [Цели и задачи](#)
4. [Структура проекта](#)
5. [Технические детали](#)
6. [Подключение оборудования](#)
7. [Установка зависимостей](#)
8. [Подробное описание кода](#)
9. [Функциональность](#)
10. [Эволюция версий](#)
11. [Запуск проекта](#)
12. [Медиаматериалы](#)
13. [Особенности реализации](#)
14. [Возможные улучшения](#)
15. [Лицензия](#)

Авторы

Проект разработан студентами 3 курса Университета ИТМО в рамках дисциплины "Интернет вещей".

Участники проекта:

- Батаргин Егор Александрович
- Медведева Даниэла Михайловна
- Березовский Артемий Сергеевич

Преподаватель: Гоначаров Алексей Андреевич

Обзор проекта

Данный проект представляет собой инновационную систему управления компьютерной мышью с использованием инерциального датчика LSM6DS33 (гироскоп/акселерометр) и тактильных кнопок на платформе Raspberry Pi. Проект разработан в рамках курса "Интернет вещей" Университета ИТМО и демонстрирует практическое применение микроконтроллеров для создания человеко-машинных интерфейсов.

Основная цель — создание "воздушной мыши" (air mouse), которая позволяет управлять курсором компьютера движениями руки в пространстве, а также выполнять клики с помощью физических кнопок. Такое устройство может быть использовано в различных сценариях: презентации, медиацентры, игровые контроллеры, а также как вспомогательное устройство для людей с ограниченными возможностями.

Цели и задачи

Основные цели:

1. Разработать аппаратную платформу на базе Raspberry Pi с датчиком движения LSM6DS33.
2. Реализовать программное обеспечение для чтения данных гироскопа и преобразования их в перемещение курсора.
3. Обеспечить эмуляцию HID-устройства (мыши) через библиотеку `zero_hid`.
4. Интегрировать тактильные кнопки для левого и правого клика.
5. Реализовать систему активации движения через сенсор касания.

Дополнительные задачи:

- Обеспечить устойчивость к дребезгу контактов кнопок (дебаунсинг).
- Реализовать веб-интерфейс для визуализации данных сенсора в реальном времени.
- Оптимизировать код для работы в асинхронном режиме.
- Создать подробную документацию и инструкции по сборке.

Структура проекта

Проект состоит из нескольких Python-скриптов, каждый из которых представляет собой определенный этап разработки или тестовый компонент.

Основные скрипты

1. **accel.py** — начальная версия скрипта для чтения данных с датчика LSM6DS33 и управления мышью через библиотеку `zero_hid`. Включает обработку сенсора касания (GPIO 4) для активации движения. Также содержит веб-сервер для визуализации данных через WebSocket.
2. **accel12.py** — улучшенная версия с добавлением кнопок левого (GPIO 17) и правого (GPIO 18) клика, программным дебаунсингом и настраиваемой чувствительностью. Введен класс `ButtonDebouncer` для устранениядребезга.
3. **accel13.py** — третья итерация с исправлением ошибок именования (`soxgyro_range` → `sox.gyro_range`), улучшенной структурой кода и разделением на логические секции.
4. **accel13-2.py** — версия 3.2 с дополнительными исправлениями и оптимизацией антидребезга на основе временных меток.
5. **accel13-3.py** — наиболее продвинутая версия с добавлением переменной чувствительности, передаваемой в WebSocket-данные, и улучшенными комментариями.

Тестовые скрипты

6. **gpio_but.py** — простой тест сенсора касания на GPIO 17 с выводом состояния в консоль. Используется для проверки корректности подключения сенсора.
7. **test_buttons1.py** — базовый тест двух кнопок (GPIO 17 и 18) с отслеживанием изменений состояния. Выводит сообщения при нажатии/отпускании.
8. **test_buttons2.py** — продвинутый тест кнопок с программным дебаунсингом и объектно-ориентированным подходом. Включает класс `ButtonMonitor` с детектированием фронтов нажатия/отпускания.

Технические детали

Оборудование

- **Микрокомпьютер:** Raspberry Pi (любая модель с GPIO, например Raspberry Pi 3/4/Zero)
- **Датчик движения:** LSM6DS33 (6-осевой инерциальный модуль: 3-осевой акселерометр + 3-осевой гироскоп)
- **Интерфейс подключения:** I²C (адрес по умолчанию 0x6B)
- **Кнопки:** 2 тактильные кнопки (моментального действия) для левого и правого клика
- **Сенсор касания:** ёмкостный сенсор или тактильный переключатель
- **Питание:** через USB-порт Raspberry Pi или внешний источник 5V

- **Соединительные провода:** папа-папа, папа-мама в зависимости от используемой макетной платы

Программное обеспечение

- **Операционная система:** Raspberry Pi OS (ранее Raspbian) с ядром Linux
- **Язык программирования:** Python 3.7+
- **Ключевые библиотеки:**
 - RPi.GPIO — низкоуровневый доступ к GPIO
 - adafruit-circuitpython-lsm6ds — драйвер для датчика LSM6DS33
 - zero_hid — эмуляция HID-устройства (мыши) через USB
 - websockets — для сетевого взаимодействия в реальном времени
 - asyncio — для асинхронного выполнения задач
 - json — сериализация данных для передачи по сети
 - signal — обработка сигналов завершения программы

Подключение оборудования

Для корректной работы проекта необходимо правильно подключить все компоненты к Raspberry Pi. Ниже приведена таблица подключения в соответствии с нумерацией BCM (Broadcom SOC channel).

Компонент	GPIO (BCM)	Физический пин (Pinout)	Назначение
Сенсор касания	4	Pin 7	Активация движения мыши
Левая кнопка	17	Pin 11	Левый клик мыши
Правая кнопка	18	Pin 12	Правый клик мыши
Датчик LSM6DS33	I ² C (SDA)	Pin 3 (SDA)	Данные гироскопа/ акселерометра
Датчик LSM6DS33	I ² C (SCL)	Pin 5 (SCL)	Тактовый сигнал I ² C
Питание датчика	3.3V	Pin 1	Питание 3.3V
Земля датчика	GND	Pin 6	Общий провод

Примечания:

- Сенсор касания подключается без подтяжки, ожидается HIGH при касании.
- Кнопки подключены с внутренней подтяжкой к питанию (PUD_UP), поэтому нажатие соответствует LOW (0).
- Датчик LSM6DS33 работает от 3.3V, не используйте 5V для избежания повреждения.

Установка зависимостей

Перед запуском проекта необходимо установить все необходимые Python-библиотеки. Выполните следующие команды в терминале Raspberry Pi:

```
# Обновление пакетного менеджера
sudo apt update
sudo apt upgrade -y

# Установка системных зависимостей
sudo apt install python3-pip python3-venv -y

# Создание виртуального окружения (опционально)
python3 -m venv venv
source venv/bin/activate

# Установка Python-библиотек
pip install RPi.GPIO adafruit-circuitpython-lsm6ds zero-hid websockets asyncio
```

Пояснение к библиотекам:

- RPi.GPIO — предоставляет API для управления GPIO-пинами Raspberry Pi.
- adafruit-circuitpython-lsm6ds — библиотека от Adafruit для работы с датчиками серии LSM6DS.
- zero-hid — позволяет эмулировать HID-устройства (клавиатуру, мышь) через USB.
- websockets — реализация протокола WebSocket для двусторонней связи в реальном времени.
- asyncio — встроенная библиотека для асинхронного программирования.

Подробное описание кода

В этом разделе представлен детальный анализ ключевых компонентов кода проекта.

Архитектура программы

Основные скрипты (`accel.py` , `accel2.py` , `accel3.py` и их производные) построены по модульной архитектуре и включают следующие компоненты:

1. **Инициализация GPIO** — настройка пинов ввода/вывода.
2. **Инициализация датчика LSM6DS33** — установка параметров измерения (диапазон, частота, фильтры).
3. **Асинхронный цикл чтения данных** — непрерывное получение показаний гироскопа.
4. **Обработка сенсора касания** — активация движения мыши только при касании.
5. **Обработка кнопок** — детектирование нажатий с дебаунсингом.
6. **Управление курсором** — преобразование угловой скорости в перемещение пикселей.
7. **Веб-сервер и WebSocket-сервер** — предоставление данных для визуализации.

Ключевые функции и классы

Функция `zero_if_small(x, threshold=0.02)`

```
def zero_if_small(x, threshold=0.02):  
    """  
    Обнуляет значения ниже порога для устранения дрейфа датчика.  
  
    Аргументы:  
        x (float): Входное значение  
        threshold (float): Порог обнуления (по умолчанию 0.02)  
  
    Возвращает:  
        float: 0.0 если |x| < threshold, иначе x  
    """  
    return 0.0 if abs(x) < threshold else x
```

Назначение: Гироскопы имеют незначительный дрейф (drift), который приводит к появлению малых значений даже при отсутствии движения. Данная функция отсекает шум, обнуляя значения ниже заданного порога. Это улучшает точность управления курсором.

Класс ButtonDebouncer (В accel2.py)

```
class ButtonDebouncer:
    """
    Программный дебаунсер для устранения дребезга контактов кнопок.
    """
    def __init__(self, pin, debounce_time):
        self.pin = pin
        self.debounce_time = debounce_time
        self.last_state = GPIO.input(pin)
        self.last_change = time.time()
        self.stable_state = self.last_state

    def read(self):
        current_read = GPIO.input(self.pin)
        current_time = time.time()

        if current_read != self.last_state:
            self.last_change = current_time
            self.last_state = current_read

        if (current_time - self.last_change) > self.debounce_time:
            if self.stable_state != self.last_state:
                self.stable_state = self.last_state

        return self.stable_state == GPIO.LOW
```

Назначение: Механические кнопки при нажатии/отпускании генерируют множественные быстрые изменения состояния (дребезг). Класс ButtonDebouncer фильтрует эти помехи, учитывая только устойчивые состояния, которые длятся дольше заданного времени `debounce_time` .

Асинхронная функция `sensor_reader()`

```
async def sensor_reader():
    """
    Основной цикл чтения данных сенсора и управления мышью.
    """
    global sox, shutdown, m

    if sox is None:
        i2c = board.I2C()
        sox = LSM6DS33(i2c, address=0x6B)
        sox.accelerometer_rate = Rate.RATE_SHUTDOWN
        sox.accelerometer_range = AccelRange.RANGE_2G
        sox.high_pass_filter = AccelHPF.HPF_DIV400
        sox.gyro_data_rate = Rate.RATE_208_HZ
        sox.gyro_range = GyroRange.RANGE_125_DPS

    while not shutdown:
        try:
            gyro = list(sox.gyro)
            # Калибровочные смещения
            gyro[0] += 0.11011872144861299
            gyro[1] -= 0.24510967682695364
            gyro[2] += 0.030543261909900768

            # Сохранение данных для WebSocket
            sensor_reader.latest_data = {
                "accel": {"x": 0, "y": 0, "z": 0},
                "gyro": {
                    "x": zero_if_small(gyro[0]),
                    "y": zero_if_small(gyro[1]),
                    "z": zero_if_small(gyro[2])
                }
            }

            # Движение мыши при касании
            touch_state = GPIO.input(TOUCH_PIN)
            if touch_state == GPIO.HIGH:
                m.move(-round(gyro[2] * 8), -round(gyro[0] * 8))

            await asyncio.sleep(0)
        except Exception as e:
            print(f"Sensor error: {e}. Reconnecting...")
```



```
sox = None
await asyncio.sleep(1)
```

Назначение: Это ядро программы, выполняющее следующие задачи:

- Инициализация датчика при первом запуске.
- Чтение гироскопических данных с применением калибровочных смещений.
- Фильтрация шума через `zero_if_small`.
- Управление курсором мыши при активации сенсором касания.
- Обработка исключений с повторной инициализацией датчика при ошибках.

WebSocket-обработчик `websocket_handler(websocket, path)`

```
async def websocket_handler(websocket, path):
    """
    Обработчик WebSocket-соединений для передачи данных сенсора.
    """
    global shutdown
    print("Client connected")

    try:
        while not shutdown:
            if hasattr(sensor_reader, 'latest_data'):
                await websocket.send(json.dumps(sensor_reader.latest_data))
                await asyncio.sleep(0.01)
    except websockets.exceptions.ConnectionClosed:
        print("Client disconnected")
    except Exception as e:
        print(f"WebSocket error: {e}")
```

Назначение: Обеспечивает передачу данных сенсора в реальном времени клиентам (например, веб-браузеру) через протокол WebSocket. Данные сериализуются в JSON и отправляются с частотой ~100 Гц.

HTTP-обработчик `http_handler(reader, writer)`

```
async def http_handler(reader, writer):
    """
    Обработчик HTTP-запросов для обслуживания HTML-страницы визуализации.
    """
    request = await reader.read(1024)
    response = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html\r\n"
        "Connection: close\r\n"
        "\r\n"
        # HTML-страница с графиками Chart.js
    )
    writer.write(response.encode())
    await writer.drain()
    writer.close()
```

Назначение: Возвращает статическую HTML-страницу с графиками Chart.js для визуализации данных сенсора. Страница подключается к WebSocket-серверу и отображает实时 графики ускорения и угловой скорости.

Принцип работы антидребезга

В проекте реализовано два подхода к антидребезгу:

1. **Временной антидребезг (в `acce13.py`)** — используется сравнение временных меток. Состояние кнопки считается измененным только если оно стабильно в течение заданного интервала (например, 50 мс).
2. **Классовый антидребезг (в `acce12.py` и `test_buttons2.py`)** — более продвинутый подход с использованием объекта, отслеживающего историю состояний. Класс `ButtonMonitor` предоставляет методы `on_press()` и `on_release()` для детектирования фронтов.

Калибровка датчика

Датчик LSM6DS33 имеет систематические ошибки (смещения нуля). В коде применяется программная калибровка путем добавления/вычитания констант:

```
gyro[0] += 0.11011872144861299
gyro[1] -= 0.24510967682695364
gyro[2] += 0.030543261909900768
```

Эти значения получены эмпирически в процессе калибровки устройства. Для каждого конкретного экземпляра датчика значения могут отличаться.

Функциональность

Управление курсором

- **Преобразование данных:** Угловая скорость по осям X, Y, Z гироскопа преобразуется в перемещение курсора по осям X и Y экрана.
- **Маппинг осей:** В проекте используется маппинг `gyro[2] → X` и `gyro[0] → Y`, что соответствует ориентации датчика на устройстве.
- **Чувствительность:** Регулируется коэффициентом `sensitivity` (по умолчанию 8). Увеличение значения делает курсор более отзывчивым.
- **Пороговая фильтрация:** Функция `zero_if_small` исключает дрейф датчика.

Обработка кнопок

- **Левая кнопка (GPIO 17):** Эмулирует левый клик мыши.
- **Правая кнопка (GPIO 18):** Эмулирует правый клик мыши.
- **Дебаунсинг:** Программная фильтрациядребезга контактов.
- **Детектирование фронтов:** Возможность определения момента нажатия и отпускания.

Сенсор касания

- **Активация движения:** Курсор перемещается только когда сенсор касания активирован (состояние HIGH).
- **Энергосбережение:** В отсутствие касания данные гироскопа игнорируются, что снижает нагрузку на систему.
- **Тактильный интерфейс:** Позволяет пользователю интуитивно контролировать активность мыши.

Визуализация данных

- **Веб-интерфейс:** Доступен по адресу `http://<IP_Raspberry_Pi>:8000`.

- **Графики в реальном времени:** Отображение акселерометра и гироскопа с использованием Chart.js.
- **WebSocket-подключение:** Данные передаются с частотой 100 Гц.

Эволюция версий

Проект развивался итеративно, каждая версия вносила улучшения и исправления.

Версия 1.0 (`ассел.ру`)

- Базовая функциональность чтения гироскопа.
- Управление курсором через сенсор касания.
- Веб-сервер и WebSocket-сервер для визуализации.
- Отсутствие обработки кнопок.

Версия 2.0 (`ассел2.ру`)

- Добавлены левая и правая кнопки мыши.
- Реализован класс `ButtonDebouncer` для антидребезга.
- Введены конфигурационные константы (`DEBOUNCE_TIME` , `GYRO_THRESHOLD` , `GYRO_SENSITIVITY`).
- Исправлена опечатка `soxgyro_range` → `sox.gyro_range` .

Версия 3.0 (`ассел3.ру`)

- Улучшена структура кода с разделением на секции.
- Антидребезг на основе временных меток.
- Более подробные комментарии на русском языке.

Версия 3.2 (`ассел3-2.ру`)

- Дополнительные исправления и оптимизация.
- Улучшена обработка кнопок.

Версия 3.3 (`ассел3-3.ру`)

- Добавлена переменная `sensitivity` , передаваемая в WebSocket.
- Улучшена документация.
- Наиболее стабильная и функциональная версия.

Запуск проекта

Подготовка оборудования

1. Соберите схему согласно таблице подключения.
2. Убедитесь, что все соединения надежны.
3. Подключите Raspberry Pi к монитору или используйте SSH для удаленного доступа.

Установка зависимостей

Выполните команды, описанные в разделе [Установка зависимостей](#).

Запуск основного скрипта

```
python accel3-3.py
```

После запуска вы увидите в терминале:

```
WebSocket server started on ws://0.0.0.0:8765  
HTTP server started on http://0.0.0.0:8000
```

Тестирование компонентов

- **Тест сенсора касания:** `python gpio_but.py`
- **Тест кнопок (базовый):** `python test_buttons1.py`
- **Тест кнопок (продвинутый):** `python test_buttons2.py`

Доступ к веб-интерфейсу

Откройте браузер на любом устройстве в той же сети и перейдите по адресу

`http://<IP_Raspberry_Pi>:8000` . Вы увидите страницу с графиками, которые будут обновляться в реальном времени.

Медиа материалы

В папке `медиа` содержатся фотографии и видео, демонстрирующие процесс сборки и работу устройства.

Фотографии сборки

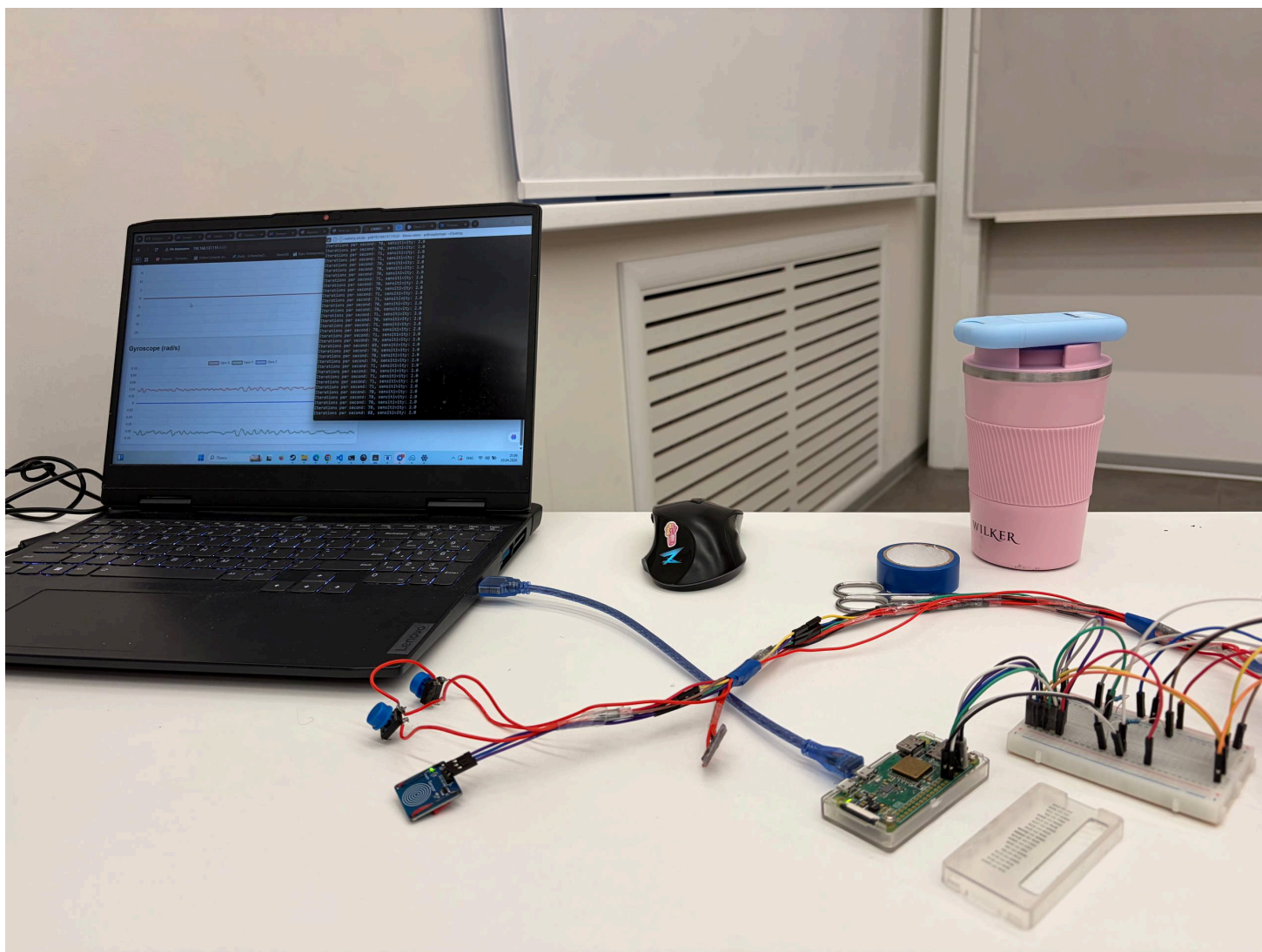
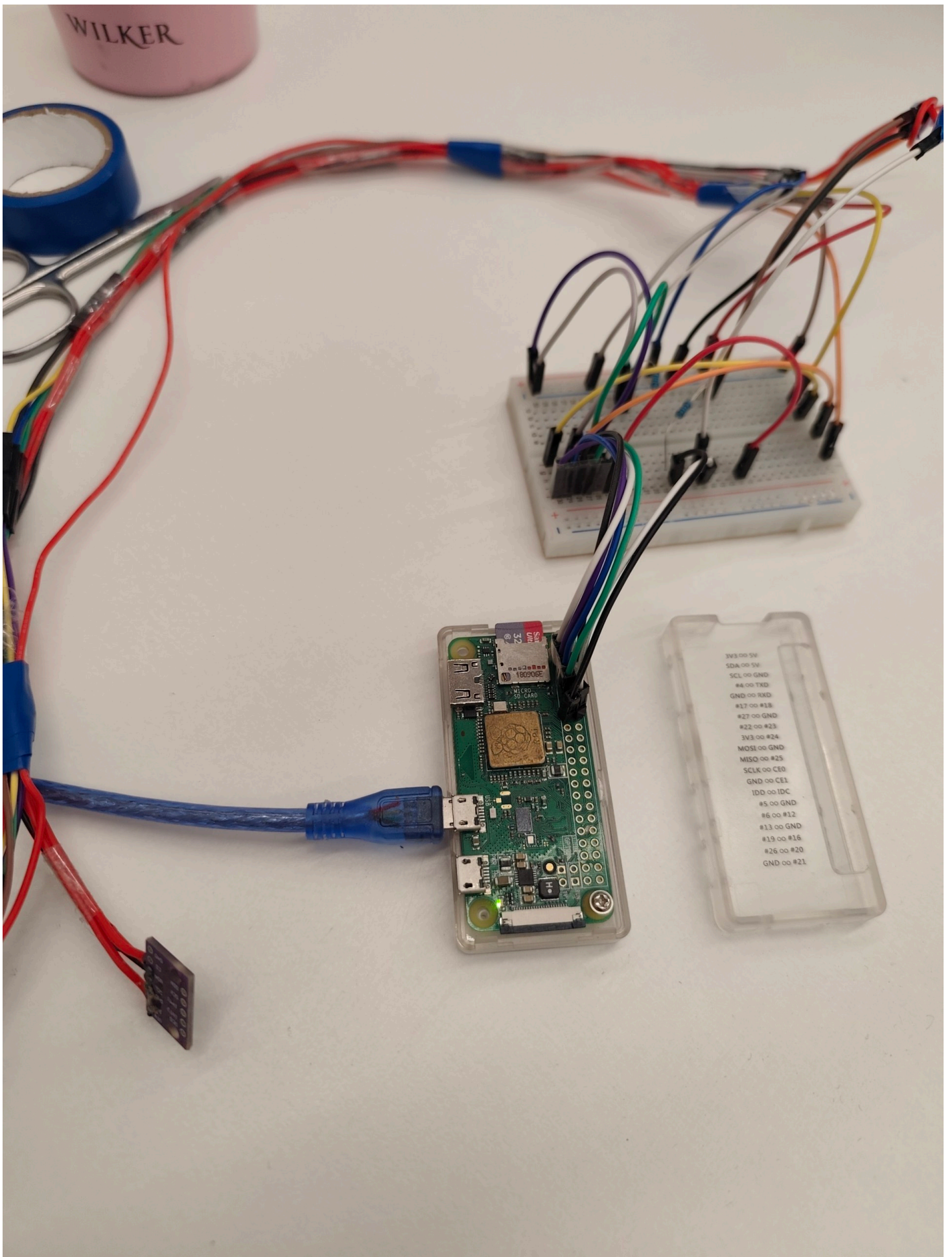
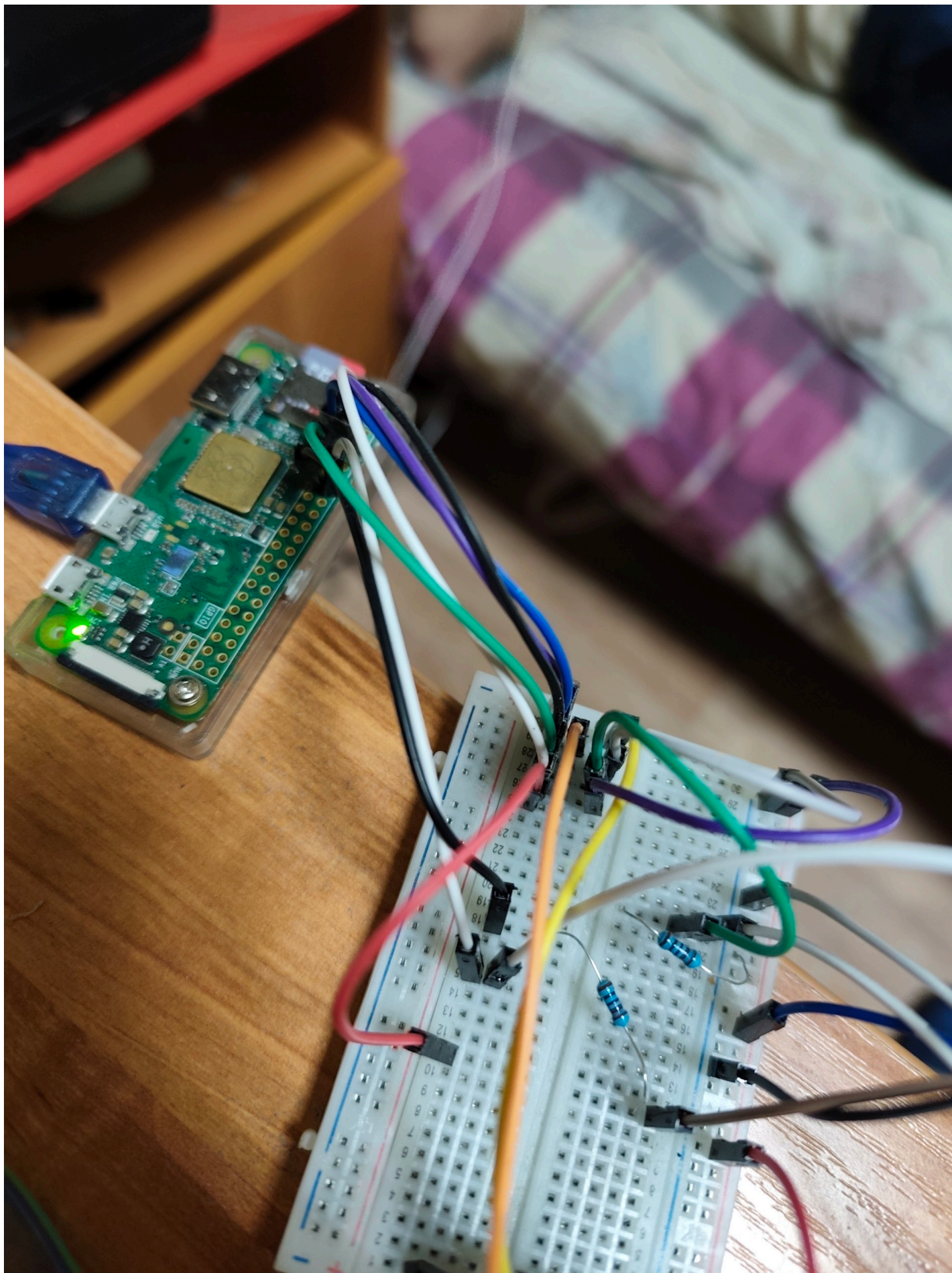


Рисунок 1: Общий вид сборки с подключением к компьютеру.



3V3 00 5V
SDA 00 5V
SCL 00 GND
#4 00 TXD
GND 00 RXD
#17 00 #18
#27 00 GND
#22 00 #23
3V3 00 #24
MOSI 00 GND
MISO 00 #25
SCLK 00 CE0
GND 00 CE1
IDD 00 IDC
#5 00 GND
#6 00 #12
#13 00 GND
#19 00 #16
#26 00 #20
GND 00 #21

Рисунок 2: Детали монтажа датчика LSM6DS33 на макетной плате.



Видео работы

0:00

Видео 1: Демонстрация управления курсором с помощью жестов и нажатия кнопок.

Описание видео: На видео показано, как оператор перемещает устройство в пространстве, и курсор на экране повторяет эти движения. Также демонстрируется работа левой и правой кнопок мыши.

Особенности реализации

Асинхронность

Использование библиотеки `asyncio` позволяет параллельно выполнять несколько задач:

- Чтение данных сенсора.
- Обработка кнопок.
- Обслуживание WebSocket-соединений.
- Обслуживание HTTP-запросов.

Это повышает отзывчивость системы и эффективно использует ресурсы процессора.

Глобальные переменные

Для упрощения кода используются глобальные переменные:

- `sox` — экземпляр датчика LSM6DS33.
- `shutdown` — флаг завершения работы.
- `m` — объект мыши `zero_hid.Mouse`.

Такой подход оправдан в небольших проектах, но в более крупных системах рекомендуется использовать классы и инкапсуляцию.

Обработка прерываний

Программа корректно обрабатывает сигналы `SIGINT` (Ctrl+C) и `SIGTERM`:

- Устанавливается флаг `shutdown = True`.
- Закрываются WebSocket- и HTTP-серверы.
- Выполняется очистка GPIO (`GPIO.cleanup()`).

Модульность

Каждый скрипт выполняет определенную функцию, что облегчает тестирование и отладку.

Тестовые скрипты (`gpio_but.py` , `test_buttons1.py` , `test_buttons2.py`) могут использоваться

независимо от основных.

Возможные улучшения

Проект имеет потенциал для дальнейшего развития. Ниже перечислены возможные направления улучшений.

1. Расширение жестового управления

- **Двойной клик:** Двойное касание сенсора.
- **Прокрутка:** Наклон устройства вперёд/назад для скроллинга.
- **Масштабирование:** Поворот устройства вокруг оси Z для zoom in/out.

2. Улучшение калибровки

- **Автоматическая калибровка:** Программное определение смещений нуля при запуске.
- **Калибровка через веб-интерфейс:** Возможность пользовательской калибровки через браузер.

3. Конфигурационный файл

- **JSON-конфиг:** Хранение параметров (чувствительность, пороги, пины) в отдельном файле.
- **Графический редактор конфигурации:** Веб-интерфейс для настройки параметров.

4. Расширенная визуализация

- **3D-модель ориентации:** Отображение положения устройства в пространстве с помощью WebGL.
- **Исторические данные:** Графики за более длительный период.
- **Экспорт данных:** Возможность сохранения данных в CSV для анализа.

5. Беспроводная связь

- **Bluetooth HID:** Использование Bluetooth для подключения к компьютеру без проводов.
- **Wi-Fi Direct:** Прямое соединение с устройством без роутера.

6. Оптимизация энергопотребления

- **Режим сна:** При отсутствии активности устройство переходит в спящий режим.

- **Динамическая частота опроса:** Автоматическое снижение частоты опроса датчика при отсутствии движений.

7. Поддержка дополнительных платформ

- **Поддержка Arduino:** Портируемость кода на платформы Arduino с использованием аналогичных датчиков.
- **Поддержка ESP32:** Использование микроконтроллера ESP32 с Wi-Fi/Bluetooth.

8. Документация и сообщество

- **Пошаговые инструкции:** Видео-туториалы по сборке.
- **Форум поддержки:** Создание сообщества для обмена опытом и решения проблем.

Благодарности и контакты

Благодарности:

- Коллективу кафедры компьютерных технологий ИТМО за предоставленное оборудование.
- Сообществу Adafruit за открытые библиотеки и документацию.
- Разработчикам библиотеки `zero_hid` за простой API эмуляции HID.

Контакты: Для вопросов и предложений пишите на email: egorbatargin@itmo.ru